Space Mission Design Game

NASA Space Apps 2026: team guide to the challenge resources, our 3 missions and how to build them in Godot

How to use this document. Section 1 lists the 3 documents everyone must read. Section 2 is what the judges want. Section 3 covers each NASA resource: what's in it and what it means for our game. Sections 4–6 apply all of that to our Moon / Mars / Saturn missions, GMAT + Godot and our game systems. Section 7 is a work plan for the 6 of us.

Numbers marked **(verify)** are well-known reference values I added from general NASA knowledge. They are *not* from the challenge resources. Check each one on the NASA mission page before putting it in the game, and record the source in our data sheet.

1. Must-read: 3 documents for the whole team

#1 Design and Develop Science Missions (NASA Small Spacecraft Systems Virtual Institute, 25 slides, ~30 min)

Why: It's the shortest resource and the most important one. It gives our game its **skeleton**: the 4 mission segments (Launch, Space, Ground, User) and NASA's process *Need & Goals → Concept → Requirements → Design → Build & Verify → Implement*. Its key idea is "begin with the end in mind": decide what the scientist needs at the end, then work backwards to the spacecraft.
Who: Everyone. Read it first.

#2 Space Mission Concept Development Using Concept Maturity Levels (JPL, Wessen et al., AIAA 2013, ~10 pages)

Why: It explains how real JPL teams judge whether a mission concept is good, and it maps almost directly onto game mechanics: the feasibility → trade space → point design loop (= our redesign-and-retry loop), the **green / yellow / red checklist** where 80% green means "done" (= our end-of-mission report card), and "science partials" (how science return changes when you change mass, data or orbit, which is exactly what our ship builder should show).
Who: Everyone, especially the game designer and whoever builds scoring.

#3 CubeSat 101 (NASA CubeSat Launch Initiative, 96 pages). Read chapters 1, 2 and 6.1–6.9 only

Why: It's the most practical "how a real small spacecraft goes from idea to orbit" guide, written for first-timers. Chapter 2 has the real development timeline and great "Free Advice" boxes that make good in-game hints and failure events. Chapter 6 covers testing (vibration, thermal vacuum, Day-In-The-Life), which can become a pre-launch testing step in the game.
Who: Everyone reads Ch. 1–2. The data/science person and the engineers also read Ch. 6.

Also play (not a PDF): NASA Webb's *Build It Yourself* telescope game, about 15 minutes. It's the closest existing example of what we're making. Play it as a team and write down what you'd copy and what you'd do better.

2. What the challenge (and the judges) actually want

The challenge asks for an interactive game where players **design, manage and simulate a complete space mission**, making engineering decisions under limited resources and seeing how each choice shapes success. The page describes a strong solution as having these parts:

Challenge asks for	How our game covers it
Mission configuration system	Craft design screen: objective → parts → launch vehicle → route
Resource management: **mass, power, budget, data, comms, time**	Always-visible resource bars that update as you add parts
Mission simulation & evaluation	The flown mission (travel, landing / driving / photographing) plus RNG events
Collaboration for up to 4 players, with roles	2-player local co-op (Fireboy & Watergirl style), with LAN as a stretch
Feedback that explains *why*, not just right/wrong	Mission report: what failed, which decision caused it, what would have worked
Accessible; science explained simply, in-game	Hint levels (Easy full, Medium partial, Hard none) and short "NASA fact" tooltips
Extras: scenarios, difficulty levels, random events, reports, leaderboards, sandbox	3 missions = 3 difficulties, RNG events, report, and leaderboard if time allows

From our notes → design rule: "Reward smart and effective engineering decisions, everything else is extra." Every feature should pass one test: *does this make a player choice matter?* If it doesn't, it's polish, so cut it first when we're short on time.

3. The resources, one by one

3.1 Design and Develop Science Missions (slides, March 2025)

- **Science mission** = a spaceflight mission focused on research in Earth science, heliophysics, planetary science or astrophysics. All 3 of our missions are planetary science.
- **4 segments.** *Launch*: the rocket and launch services. *Space*: the spacecraft, orbiters, landers, rovers and instruments. *Ground*: ground stations and mission control. *User*: the scientists and public who use the data.
- **Process:** Identify Need & Goals → Define Concept → Define Requirements → Design → Build & Verify → Implement.
- **Concept of Operations (ConOps):** a time-ordered story of how the system will be used. It drives the requirements and is the yardstick for "did it work?".
- **Requirements are numbers, and each one has a verification method.** The example mission (BioNutrients-2) had requirements like "12 samples", "< 250 g container" and "< 300 mL volume", and the 250 g limit is verified by weighing the container.
- The deck also lists example missions (PUNCH, Dawn, a Phobos/Deimos sample return concept, SLS, Artemis) and references the NASA Systems Engineering Handbook.

For our game:

- Turn the 4 segments into the 4 phases of every level: **Launch** (pick a rocket), **Space** (build the craft), **Ground** (comms and antenna), **User** (the end report scores the science delivered).
- Show each mission's requirements as a short checklist with real numbers, e.g. "Land below 2 m/s", "Return 1 rock sample", "Send 20 photos to Earth". The game then **verifies** each one, just like the slide.
- Start every level with a 3–4 panel ConOps comic: "Launch → cruise → arrive → do science → send data". It teaches the idea in 10 seconds.

3.2 Concept Maturity Levels (JPL paper, 2013)

CML	Meaning (summarized)	Game use
1 Cocktail Napkin	Science question, type of observations, rough sketch, what's unique about the idea	Pick the mission objective
2 Initial Feasibility	Basic calculations show whether it's viable at all	Live budget bars turn red if the design is impossible
3 Trade Space	Explore options to see how they affect science return, cost and risk	Compare 2 saved designs side by side
4 Point Design	One chosen design, defined to major subsystems, with margins and reserves	"Freeze design & launch"
5–9	Baseline, integrated concept, then preliminary and critical design reviews (formal NASA project stages)	Flavor text only

- **Iteration is normal:** teams loop between CML 2, 3 and 4, and going back to trade space after hitting a problem is a *strength*, not a failure. Teams that skipped trade studies ended up with lower science and inefficient designs.
- **Checklist scoring:** each item is green (done), yellow (in progress) or red (not started). A concept counts as reaching a level at about **80% green**, but engineering judgment still applies: missing a critical item outweighs a high percentage.
- **Self-assessment is too optimistic,** so the paper recommends an independent reviewer.
- Cost classes are mentioned: **Explorer, Discovery, New Frontiers, Flagship**.

For our game:

- **End report = a CML-style checklist** with green/yellow/red per subsystem (Power, Comms, Propulsion, Science, Budget, Margins). Some items are "critical": failing one fails the mission even at 90% green, exactly like the paper says.
- **Budget tiers** by difficulty: Moon = Discovery-class, Mars = New Frontiers-class, Saturn = Flagship-class. These are real NASA program names, and it's a nice touch to show them.
- **Co-op idea:** player 2 is the "independent reviewer" and must approve player 1's design before launch, as the paper recommends.
- **Reward margins:** CML 4 requires "acceptable margins and reserves". Give a reliability bonus for keeping ~10–20% spare mass and power, and RNG events hurt less with margin.

3.3 CubeSat 101 (NASA CSLI, 2017)

- **Sizes:** a 1U is a 10 cm cube of about 1–1.33 kg. Common sizes are 1.5U, 2U, 3U, 6U and 12U. A "small satellite" is anything under 300 kg.

- **Real development timeline** (great for a "development time" resource): concept 1–6 months, funding 1–12, design 1–6, waiting for a launch slot 1–36, licensing 4–6, ground station 2–12, hardware build & test 2–12, readiness review half a day, launch 1 day, operations up to 20 years.
- **Money:** NASA's launch initiative covers launch only (up to about $300k, typically enough for a 3U to low Earth orbit). Teams pay for everything else, and the guide advises **at least a 10% budget reserve**.
- **"Free Advice" boxes:** keep it simple; don't design to the exact size limit; something will break during vibration testing, so put important parts where they're easy to reach; use two burn wires for deployables; prefer parts that have flown before ("flight heritage"); use certified batteries.
- **Ground station:** "Launching with an inadequate ground station is a mission killer." Build and test it early.
- **Testing:** vibration/shock, thermal vacuum bakeout, and "Day In The Life" (DITL), which checks that timers, deployments and the first radio contact work with the final software.
- **Orbital debris:** you must show your spacecraft won't endanger other spacecraft, will deorbit in reasonable time, and won't leave large pieces surviving reentry. Prefer low-melting-point materials like aluminum over titanium or tungsten.
- **Team lesson:** projects lose knowledge when members leave, so document everything. That's a good lesson for us too.

For our game:

- **"Flight heritage" parts:** cheaper and more reliable but lower performance. **"New tech" parts:** better stats but a higher chance of RNG failure. This is a real NASA trade-off and very easy to implement.
- **Optional pre-launch test step:** spend budget and time on vibration, thermal or DITL tests. Each test you run reveals and fixes hidden faults before launch, and each one you skip may become an RNG failure in flight. This makes "cruel but rewarding" concrete.
- **Hint text:** use the Free Advice boxes for Easy mode hints, credited to "NASA CubeSat 101".
- **The ground station matters:** a weak antenna or ground link means photos and data never arrive (Saturn level).

3.4 Mission Design Lab (NASA Goddard)

- A collaborative environment where engineers rapidly turn ideas into assessed mission designs.
- **Study types:** Office Hours (~1 hour of brainstorming), Concept Development (a few half-days: feasibility and risk), Tailored Trade Study (concept, ConOps, resource summary, trade tree, rough cost), Point Design (6 days: CAD, Master Equipment List, engineering models, cost estimate).
- **16 disciplines:** attitude control, mission cost, propulsion, avionics, mechanical design, reliability, flight dynamics, flight software, mechanical systems, orbital debris, radiation environment, structural, communications, mission operations, power systems, thermal.
- Covers end-of-mission planning, including landing and **sample return**, which is relevant to our Mars rock mission.

For our game: use the disciplines as our **parts categories** and **co-op roles**. Simplified: P1 = Flight Dynamics & Propulsion (route, fuel, landing); P2 = Power, Comms & Science (panels/RTG, antenna, instruments). Show a "Master Equipment List" (every part with mass, power and cost) on the design screen. It's what real teams use, and it's basically our inventory.

3.5 GMAT (General Mission Analysis Tool)

- NASA's open-source mission design, optimization and navigation software, for Windows, Linux and macOS, downloaded from SourceForge. It handles low Earth orbit, lunar, libration-point and deep-space/interplanetary missions.

- Does orbit propagation, maneuver planning, launch-window analysis and trajectory optimization, with plots and reports.

> **Important for us:** GMAT is a **standalone desktop program**. It can't run inside Godot as a real-time physics engine. The right way to "use GMAT physics" is: **run GMAT offline → export the results as tables (ReportFile output, CSV/text) → load them in Godot.** See Section 5 for the pipeline. For the player-controlled landing, use Godot's own 2D physics with the real gravity value of each world.

3.6 NASA Mission Viz (WSMV)

- NASA web demos and tutorials for showing orbits: solar-system orreries, a table of **planetary orbital parameters**, an Earth-to-Moon transfer animation, and a tutorial on turning a 2D ellipse into a 3D orbit with a simple orbit propagator.

> **For our game:** use its planetary orbital parameters to place the Moon, Mars and Saturn correctly on our 2D travel map, and its Earth–Moon visualization as a reference for the Easy level's cruise screen. The ellipse tutorial explains the math for drawing 2D orbits in Godot.

3.7 Build It Yourself (NASA Webb telescope game)

- Flow: choose the science → choose wavelengths, instruments and optics → launch → see your satellite → learn which **real missions** collected similar data.
- Mechanics: limited picks per level force you to prioritize; impossible options are grayed out; choices can conflict; you can change things freely until launch; 3 levels of increasing size.
- Weaknesses noted on its own page: no built-in accessibility features, and not designed for phones.

> **For our game:** copy the "science first, then hardware" flow, "reversible until launch", and especially **"your design is like real mission X"** at the end (e.g. "Your lander is closest to Chandrayaan-3"). Beat it on accessibility: readable fonts, colorblind-safe bars, and keyboard and controller support.

3.8 Visualize a Space Mission Using VR (Space Apps 2023 challenge)

A past challenge on showing a whole mission: concept, trajectory, **timeline of events** and 3D spatial understanding. **For us:** add a mission timeline bar (Launch → Cruise → Arrival → Science → End), and look at that year's winning projects so ours feels different.

3.9 CSA (Canadian Space Agency) digital games and activities

A partner-agency collection of games and activities. Worth 20 minutes for inspiration. Citing a partner-agency resource in our submission may also count for partner-data recognition.

4. Our 3 missions: real data, trade-offs and RNG events

All values in this section are reference values to **verify** on NASA mission pages before use. They come from general NASA knowledge, not from the challenge resources.

EASY Moon: land a spacecraft safely

Fact (verify)	What it means in the game
Gravity 1.62 m/s² (about 1/6 of Earth's)	Use this as the gravity in the Godot landing scene
No atmosphere	No parachutes or heat shield. The landing is all rocket engine, so fuel is everything.
About 384,400 km away; ~3 days of travel	A short cruise, so few RNG events
Signal delay ~1.3 s each way	The player can steer the landing live, which fits Easy
A lunar day/night cycle is ~29.5 Earth days (~2 weeks of darkness)	Solar-powered landers need to land in daylight, or carry big batteries and heaters

Key trade-off: more fuel means a softer, safer landing, but more mass means you need a bigger, more expensive rocket. Landing legs, cameras and altimeters add mass but make the landing easier.

Real stories to use as RNG events or lessons (verify details):

- **Apollo 11 (1969):** computer overload alarms during descent, and the crew landed with very little fuel to spare. *Event: "Computer overload! Better avionics = no effect."*
- **Beresheet (Israel, 2019):** a sensor glitch and an engine shutdown during descent led to a crash. *Event: "Sensor fault: backup sensor (redundancy) saves you."*
- **IM-1 Odysseus (2024):** a laser rangefinder problem; it landed but tipped onto its side. *Event: "Altimeter failure: land using the camera only."*
- **Chandrayaan-3 (India, 2023):** a successful landing after its predecessor crashed. A good "learn from failure" example for the end report.

MEDIUM Mars: land a rover, drive to a rock and recover a sample

Fact (verify)	What it means in the game
Gravity 3.71 m/s² (about 0.38 g)	Gravity for the landing and driving scenes
Thin atmosphere (~1% of Earth's pressure)	You need a heat shield + parachute + rockets. It's too thin to slow you fully but thick enough to burn you up. This is the "7 minutes of terror".
A launch window about every 26 months	A development-time deadline: miss it and you pay to wait
Cruise about 7 months (Perseverance: July 2020 → Feb 2021)	A longer cruise means more RNG events
Signal delay roughly 3–22 min each way	**No live control during landing.** The player programs the landing sequence in advance, a great new mechanic.
Rover power: solar (Spirit, Opportunity) or a nuclear battery (Curiosity, Perseverance)	Solar is cheaper but vulnerable to dust; nuclear is expensive and steady

Perseverance stores rock cores in sealed sample tubes for a future Mars Sample Return	Our "rescue a rock" = drill a core, seal it, bring it to the pickup point

Key trade-offs: a bigger parachute or heat shield is safer but adds mass; solar vs. nuclear power; a better drill or arm means a better sample but more power and mass; faster driving means more risk of getting stuck.

Real stories to use as RNG events or lessons (verify details):

- **Mars Climate Orbiter (1999):** lost because one team used imperial units and another used metric. *A perfect educational event.*
- **Opportunity (ended 2018–19):** a global dust storm covered its solar panels. *Event: "Dust storm: solar output drops to 10%."*
- **Spirit (2009):** got stuck in soft sand and never got free. *Event: "Sand trap: was your route scanned?"*
- **Schiaparelli (ESA, 2016):** a sensor gave a wrong reading during descent, the lander thought it had landed, and it shut off its engines too early.

HARD Saturn: reach Saturn and photograph it

Fact (verify)	What it means in the game
About 9.5 times farther from the Sun than Earth; sunlight is ~1% of Earth's	Solar panels basically don't work, so the player needs an RTG (nuclear power): rare, expensive and heavy
Cassini: launched 1997, arrived 2004 (~7 years) via Venus, Venus, Earth, Jupiter flybys	A route choice: direct (impossible fuel) vs. gravity assists (slow but possible)
Signal delay roughly 70–85 min each way	The craft must be autonomous. A better computer means fewer missed photos.
Very low data rate at that distance	The **photo objective becomes a data problem**: bigger antenna, better compression, or fewer, sharper photos
Extreme cold	Heaters are required, and they use power
Cassini ended in 2017 by diving into Saturn, to avoid contaminating its moons	A nice "end-of-mission planning" bonus objective

Real stories to use as RNG events or lessons (verify details):

- **Huygens probe radio problem (found 2000):** the relay receiver couldn't handle the Doppler shift. Engineers saved the mission by changing the trajectory. *Event: "Comms fault discovered mid-cruise: replan the route."*
- **Huygens landing (2005):** one of its two radio channels was never switched on because of a missing command, so half the planned images were lost. *Lesson: redundancy and command checklists.*
- **Cassini's "The Day the Earth Smiled" (2013):** a famous photo of Saturn with Earth as a tiny dot. Show it as the real NASA image next to the player's photo.

Progression check: each mission should introduce one new main idea. Moon = **fuel vs. mass**. Mars = **atmosphere and signal delay (you can't steer live)**. Saturn = **power and data at extreme distance**. If a level doesn't teach its idea, simplify it until it does.

5. GMAT + Godot: how to actually connect them

1. **In GMAT (offline, one person):** model a simple spacecraft for each mission: Earth → Moon transfer, Earth → Mars transfer, and a Cassini-like gravity-assist route to Saturn. Record the **delta-v** (how much speed change the engines must give), the **trip time**, the arrival speed, and for Mars the launch window dates.
2. **Export:** GMAT's ReportFile writes chosen values to a text table. Use it to output a trajectory (time, x, y) for our 2D map animation and a summary table per route. GMAT also has scripting and an API, so check the GMAT docs if you want to automate it.
3. **Convert** the outputs to a small `missions.json` / `routes.csv` in our repo, with a "source: GMAT run, date, settings" note.
4. **In Godot:** load it with `FileAccess` + `JSON.parse_string()`. The cruise map plays back the GMAT trajectory points. The design screen uses the delta-v to check whether the player's craft carries enough fuel.
5. **Landing and driving scenes:** use Godot's own 2D physics (`RigidBody2D` or `CharacterBody2D`) with each world's real gravity (Moon 1.62, Mars 3.71 m/s^2, scaled to pixels). That way it's realistic and still fun to control.

> **Tell the judges:** "Trajectories and fuel requirements were computed with NASA GMAT; landing physics use real surface gravity." That's a clear, honest NASA-data statement.

Simple formulas we can use (keep them visible in tooltips)

```
Fuel needed (rocket equation): delta_v = Isp * 9.81 * ln(mass_full / mass_empty)
```

```
Solar power: P = 1361 W/m2 * panel_area * efficiency(~0.3) / (distance_from_sun_in_AU)^2 * sunlight_fraction
```

```
Battery: energy_needed = power_used * time_in_darkness (must be at most the battery capacity)
```

```
Data: data_sent = data_rate * contact_time; lost_photos = data_taken - data_sent - storage
```

1361 W/m^2 is the standard solar constant at Earth's distance. Dividing by distance^2 is why panels fail at Saturn, and the player sees it happen.

6. Game systems checklist

6.1 Every part is a Godot Resource (a data file)

Make a `Part` class (`extends Resource`) with `@export` variables: `mass_kg, cost, power_draw_w, power_gen_w, data_rate, science_value, reliability, heritage (bool), category, source_note`. Save each part as a `.tres` file, or generate them from a shared spreadsheet → CSV → import script. Then non-programmers can balance the game without touching code, and the `source_note` column becomes our NASA-data list.

6.2 Hints by difficulty (from our notes)

Mode	Hints
EASY	Full hints: a mentor character explains each resource bar, warns before bad choices, and quotes the CubeSat 101 Free Advice boxes
MEDIUM	Partial hints: warnings only after a mistake, or hints that cost score points
HARD	No hints. Only the end report explains what went wrong.

6.3 RNG events: random but fair

- Draw each event's **chance** from the design: low reliability, untested parts or no margin mean a higher chance.
- Draw each event's **damage** from the design: redundancy, margin or the right part means less damage.
- So the same event can be a disaster for one design and a non-issue for another. That's how "smart engineering is rewarded" even with randomness.
- Base each event on a real mission story from Section 4, and show "This really happened to ___" in the report.

6.4 End-of-mission report (most important for judging)

- A CML-style green / yellow / red checklist per subsystem, with critical items marked.
- The requirements list from the level start, each one verified or failed, with the reason.
- "What you could have done better": the top 2 changes that would have most improved the result, computed by re-scoring with alternative parts.
- "Your design is closest to real mission ___", plus a real NASA photo or fact.
- A score that rewards mission success, science returned, budget left and margins kept. Leave room for more than one good answer.

6.5 Multiplayer

- **MVP: local co-op on one keyboard** (Fireboy & Watergirl style). Use Godot's Input Map with separate actions per player (WASD vs. arrow keys). Each player owns different subsystems and both must approve launch.
- **Stretch: LAN** with Godot's high-level multiplayer (`ENetMultiplayerPeer`). Do this only after the single-player loop is finished.
- Co-op moments: Mars landing (one player handles the parachute timing, the other the engines); Saturn (one manages power, the other decides which photos to send).

6.6 Scope notes on ideas from our list

- **Crew ("how many people it holds") and returning to Earth:** these add life support and reentry, two whole extra systems. Keep all 3 missions robotic for the hackathon, and mention crewed versions as future work.
- **"GTA-style" branching outcomes:** get this from the design choices plus RNG events plus 2–3 ways to win per level (e.g. Saturn flyby vs. orbiter). No branching story needed.

7. Work plan for the 6 of us

Role	Owns
Physics / sim programmer	Landing and driving physics, cruise playback of GMAT data, RNG event system
Systems programmer	Part Resources, design screen, resource bars, scoring and report logic
UI / multiplayer programmer	Menus, HUD, hint system, 2-player input, LAN as a stretch
Artist (Blender → pixel sprites)	Parts, crafts and scenery. Tip: render Blender models with an orthographic camera at low resolution, no anti-aliasing and a limited palette for a consistent 8-bit style.
Data & science lead	The parts spreadsheet with sources, GMAT runs, verifying every number marked (verify), writing hint and fact text
Producer / pitch / QA	Playtesting with non-experts, demo video, slides, submission page and NASA data list

Build order (cut from the bottom if we run out of time)

1. **Read week:** everyone reads the 3 must-reads and plays Build It Yourself. The data lead starts the parts sheet.
2. **Core loop on the Moon:** design screen → launch check → landing → report. It must be fully playable before anything else.
3. GMAT data in for all 3 routes, and the cruise map.
4. RNG events (3–4 per mission, based on real stories).
5. Mars level: pre-programmed landing, rover drive and sample.
6. 2-player local co-op.
7. Saturn level: RTG, route choice, photo vs. data downlink.
8. Polish: hints, accessibility, leaderboard, LAN.

> **Golden rule from CubeSat 101:** keep it simple, test early and often, and document everything, so the team never loses knowledge when someone is busy.

8. Sources

Challenge resources (scanned for this guide):
Design and Develop Science Missions, NASA Small Spacecraft Systems Virtual Institute webinar slides, March 26, 2025: nasa.gov/wp-content/uploads/2025/04/250323-design-and-develop-science-mission.pdf
Wessen, Borden, Ziemer, Kwok, "Space Mission Concept Development Using Concept Maturity Levels," AIAA SPACE 2013, JPL: dataverse.jpl.nasa.gov (hdl:2014/44299)
CubeSat 101: Basic Concepts and Processes for First-Time CubeSat Developers, NASA CSLI, Oct 2017: nasa.gov (nasa_csli_cubesat_101_508.pdf)
NASA Goddard Mission Design Lab: etd.gsfc.nasa.gov/capabilities/integrated-design-center/mission-design-lab/
GMAT, NASA software GSC-19468-1: software.nasa.gov/software/GSC-19468-1 (download: sourceforge.net/projects/gmat)
NASA Mission Viz (WSMV): nasa.github.io/mission-viz/
Build It Yourself, NASA Webb: science.nasa.gov/mission/webb/game-build-it-yourself/
Space Apps 2023 "Visualize a Space Mission Using Virtual Reality" challenge; CSA Digital Games and Activities (both linked from the challenge Resources tab)

To look up for the (verify) values: NASA mission pages for Apollo 11, Chandrayaan-3, IM-1, Perseverance, Opportunity, Spirit, Mars Climate Orbiter, Cassini and Huygens; NASA Deep Space Network pages.